

JAVA 程序设计



授课人：何毅
办公室：教学楼1909



CITY INSTITUTE
城市学院

第14讲 内容向导

14.1 线程概念

14.2 创建线程

14.3 线程状态与控制

14.4 线程同步



14.1 线程概念

程序：是一段静态的代码

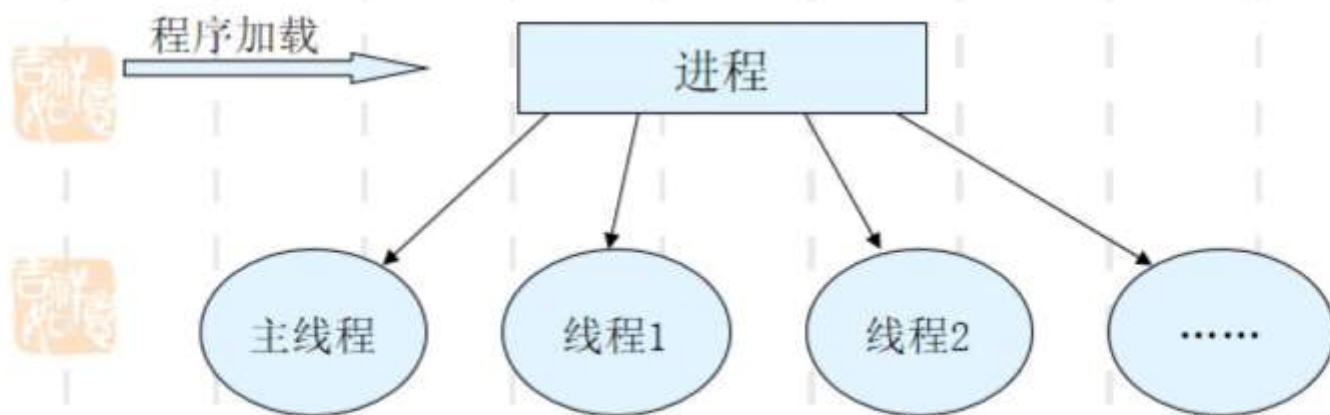
进程：程序的动态执行过程

线程（Thread）：

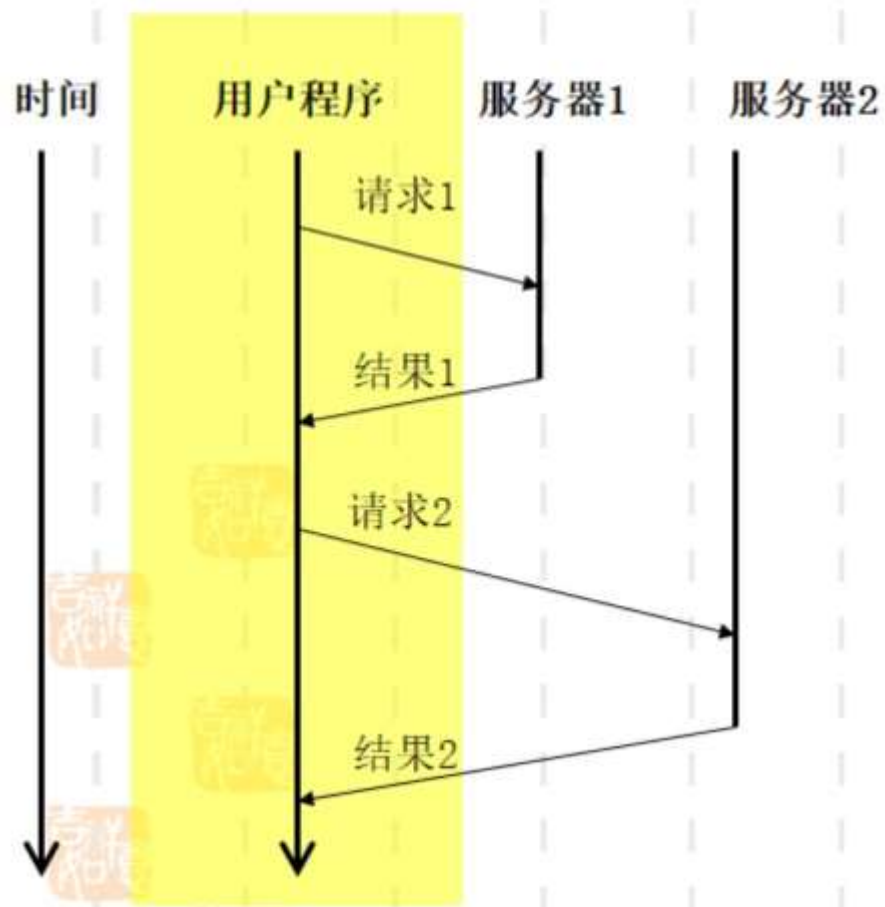
“进程”中单一顺序的控制流

多线程（multithreading）：

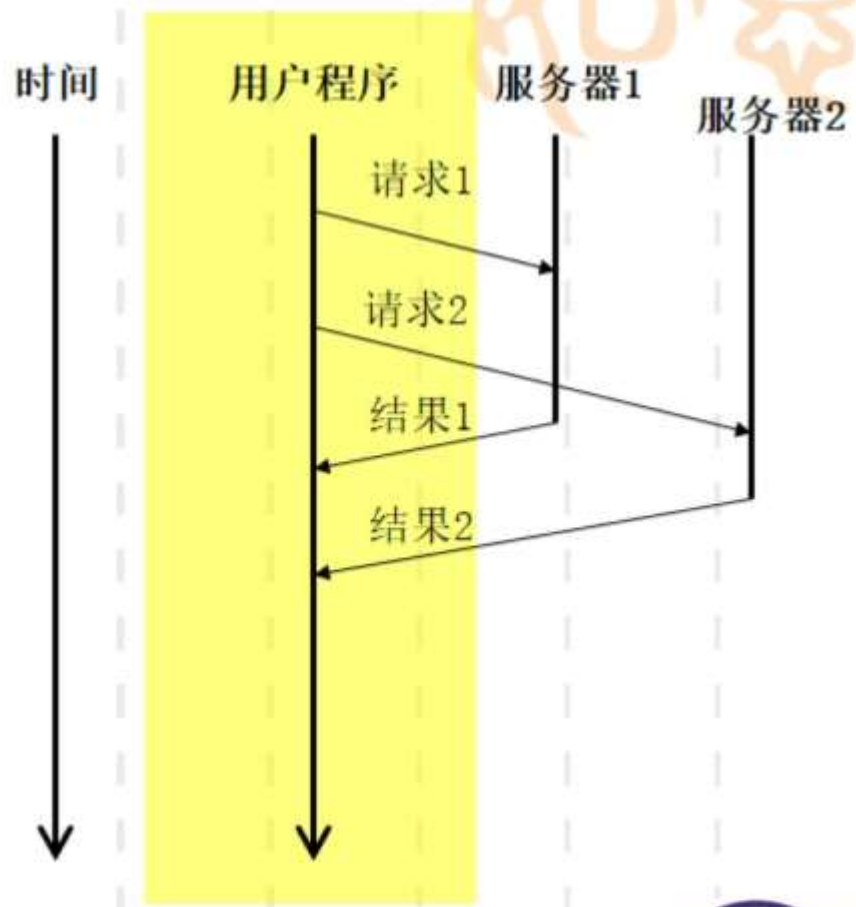
计算机同时运行多个执行线程的能力



单线程



多线程



14.2 创建线程

1. 实现线程方法

➤ 继承Thread类

直接定义Thread子类，覆写run方法，在方法里定义所要执行的具体操作

➤ 实现Runnable接口

只定义了run接口方法，在类定义时实现run方法



2.Thread类

➤创建对象

//创建线程对象

```
Thread( );
```

//制定名称

```
Thread(String threadname);
```

//线程目标对象，名称

```
Thread(Runnable target,  
String threadname);
```



➤主要方法

`start()` : 启动线程，从新建状态进入就绪排队状态

`run()` : 定义线程操作，需要定义run方法



➤ 步骤:

- ① 覆写run方法;
- ② 创建用户线程实例;
- ③ 调用start方法启动线程。

```
public class Mythread1 extends Thread {  
    public static int s_count=0;  
    private String m_name;  
    Mythread1(String s) { m_name=s;}  
    public static int getCount() { return s_count++; }  
    public void run() { ← ①  
        while(s_count<100) {  
            System.out.println(m_name+": "+getCount());  
        }  
    }  
    public static void main(String[] args) {  
        ② ← new Mythread1("first").start(); ← ③  
        new Mythread1("second").start();  
        new Mythread1("third").start();  
    } }  
}
```



3. Runnable接口

➤ 步骤:

- ①实现run方法;
- ②将该类对象作为参数创建类Thread的对象;
- ③调用start方法启动线程。

```
public class Mythread2 implements Runnable {  
    public static int s_count=0;  
    public static int getCount() { return s_count++; }  
    public void run() {  
        while(s_count<100) {  
            System.out.println(Thread.currentThread().getName()+  
                                ": "+getCount());  
        }  
    }  
    public static void main(String[] args) {  
        new Thread (new Mythread2(), "first").start();  
        new Thread (new Mythread2(), "second").start();  
        new Thread (new Mythread2(), "third").start();  
    }  
}
```



例子1:下面程序的执行过程怎样?

```
public class less{  
    public static void main(String[] args){  
        Runnable printA=new PrintChar('A',5);  
        Runnable printB=new PrintChar('B',5);  
        Runnable print100=new PrintNum(8);  
  
        Thread th1=new Thread(printA);  
        Thread th2=new Thread(printB);  
        Thread th3=new Thread(print100);  
  
        th1.start();th2.start();th3.start();  
    }  
}
```



■ **class PrintChar implements Runnable{**
 ■ **private char c;**
 ■ **private int t;**
 ■
 ■ **public PrintChar(char cc,int tt){**
 ■ **c=cc;**
 ■ **t=tt;**
 ■ **}**
 ■ **public void run(){**
 ■ **for(int i=0;i<t;i++)**
 ■ **System.out.print(c+" ");**
 ■ **}**
 ■ **}**



class PrintNum implements Runnable{
private int num;

public PrintNum(int n){
num=n;
}
public void run(){
for(int i=0;i<=num;i++)
System.out.print(i+" ");
}
}



➤ 两种创建线程方式的比较

- 直接继承**Thread**类：该类具有**Thread**类声明的方法，其对象本身就是线程对象，可以直接控制和操作。但该方式仅适用于单继承，不适用于多重继承。
- 实现**Runnable**接口：该对象本身并不是线程对象，还需要与一个**Thread**线程对象绑定在一起，作为线程对象的目标对象来使用。适用于多重继承。

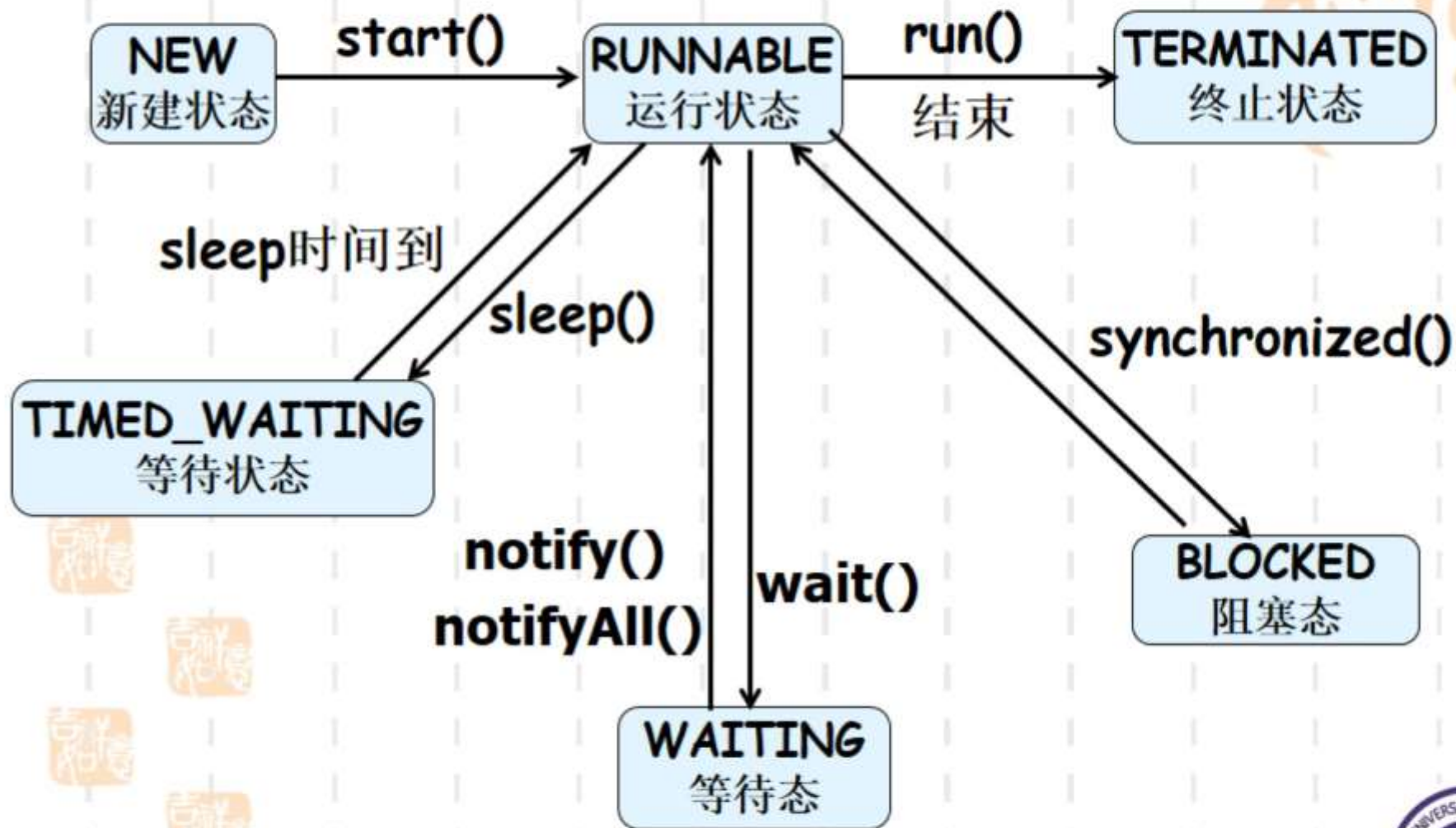


11.3 线程状态与控制

1. 线程状态

- 新建态 (**NEW**) : 使用**new**创建一个线程对象后, 它即处于新建态, 此时系统还未为它分配资源。
- 可运行态 (**RUNNABLE**) : 处于新建态的线程调用**start()**方法后, 就可进入可运行态, 等待被**CPU**调度。
- 阻塞态 (**BLOCKED**)、等待态 (**WAITING**) 和定时等待 (**TIMED_WAITING**) : 处于可运行态的线程因某种原因不能继续运行的状态, 只有当引起阻塞的原因被消除, 或等待的条件满足时, 线程才转入可运行态。
- 终止态 (**TERMINATED**) : 线程正常或异常结束后即进入该状态。







■ sleep (int millisecond)

使线程暂时休眠

■ setPriority(int newPriority)

设置线程优先级



- class less1 implements Runnable {
- public void run() {
- for (int i = 0; i < 3; i++) {
- try {
- Thread.sleep(2000);
- } catch (Exception e) {
- e.printStackTrace(); }
- System.out.println(Thread.currentThread().getName() + i); } }
- public static void main(String[] args) {
- less1 he = new less1();
- Thread demo = new Thread(he, "线程");
- demo.start(); } }



➤ 线程优先级

- **MIN_PRIORITY**, 代表最小优先级, 通常为1;
- **NORM_PRIORITY**, 代表普通优先级, 缺省为5;
- **MAX_PRIORITY**, 代表最高优先级, 通常为10。





例子:闪烁的文字

- `import java.awt.*;`
- `import javax.swing.*;`
- `class MythreadFrame extends JFrame`
- `implements`
- `Runnable{`
- `JLabel lab=new JLabel`
- `("文字闪烁",JLabel.CENTER);`
- `MythreadFrame(){`
- `super("线程测试");`
- `setLayout(new FlowLayout());`
- `add(lab);`
- `setBounds(200,200,100,100);`
- `setVisible(true);`
- `new Thread(this).start();`
- `}`





```
public void run(){
while(true){
try{
while(true){
if(lab.getText()==null)
lab.setText("文字闪烁");
else
lab.setText(null);

Thread.sleep(200);
}
} catch (InterruptedException e){
e.printStackTrace();
}}
```



■ **public class** less{
■ **public static void** main(String[] args){
■ **new** MythreadFrame();
■ }
■ }



14.4 线程同步

➤ 资源竞争

- 在多线程的环境中,可以同时做多件事情,但是,如果两个或者多个线程同时使用同一个受限资源的时候,就会出现资源访问的冲突,例如两个线程试图同时访问同一个售票系统,或同一个银行帐户等.



➤ 同步机制解决资源竞争问题

- 如果并发执行的多个线程间需要共享资源或交换数据，则这一组线程称为交互线程。
- 交互线程间存在两种关系：竞争关系和协作关系。前者需用线程互斥方式解决共享资源冲突问题；后者需要用线程同步方式解决线程间因执行速度不同而引起的不同步问题。
- 线程同步机制包括互斥和线程同步，线程互斥是线程同步的特殊情况。



■ 并发执行的交互线程间存在与时间有关的错误

- 交互的并发线程执行时，由于它们在不同时刻对同一个共享变量进行操作，线程之间相互影响、相互干扰，因此计算结果往往取决于这一组并发线程的相对速度，各种与时间有关的错误就可能出现。
- 例如：模拟四个售票点代售火车票。



■ 线程间的竞争关系会出现两个问题

- **死锁**：一组线程如果都获得了部分资源，还想要得到其他线程所占用的资源，最终所有的线程将陷入死锁。例如：哲学家进餐问题。
- **饥饿**：一个线程由于其它线程总是优先于它而被无限期拖延。例如：一个线程的优先级总是低于当前及后来线程的优先级。



■ 线程互斥和临界区管理

- 线程互斥：是解决线程间竞争关系的手段。它指若干个线程要使用同一共享资源时，任何时刻最多允许一个线程去使用，其它要使用该资源的线程必须等待，直到占有资源的线程释放该资源。
- 临界资源：共享变量代表的资源。
- 临界区：并发线程中与共享变量有关的程序段



■ 操作系统对共享同一个变量的若干线程进入各自临界区的调度原则:

- 一次至多一个线程能够在它的临界区内。
- 不能让一个线程无限期地留在它的临界区内。
- 不能强迫一个线程无限地等待进入它的临界区。特别地，进入临界区的任一线程不能妨碍正等待进入的其它线程的进展。
- 简而言之：无空等待，有空让进，择一而入，算法可行。



➤ Java的线程互斥实现

- Java提供**synchronized**关键字用于声明一段程序为临界区，使线程对临界资源采用互斥使用方式。**synchronized**的两种用法：

① 使用**synchronized**声明一条语句为临界区，该语句称为同步语句。

synchronized(对象)

语句



■ 使用**synchronized**声明同步语句，同步语句的执行过程如下：

- 当第一个线程进入临界区执行<语句>时，它获得临界资源即指定<对象>的使用权，并将对象加锁，然后执行语句对对象进行操作。
- 在此过程中，若有第二个线程也希望对同一个对象执行这条语句，由于作为临界资源的对象已被锁定，则第二个线程必须等候。
- 当第一个线程执行完临界区语句，它将释放对象锁
- 之后，第二个线程才能获得对象的使用权并运行。

这样，对于同一对象，任一时刻只能有一个线程执行临界区语句，就实现了多个线程并发地、互斥使用同一临界资源。



② 声明同步方法

- **synchronized** 方法声明
- 同步方法的方法体成为**临界区**，互斥使用(锁定)的是调用该方法的对象。该声明与以下声明效果相同：

- **synchronized (this){**

方法体

}



```
import java.awt.*;
import java.awt.event.*;
class Receiver{
    public synchronized void call(String CallerName,String msg){
        int tim;
        System.out.println("我在接"+CallerName+"打来的电话，他对我说： "
            +msg+",我在和他交谈。。。");
        try{
            tim=(int)(Math.random()*2000+1000);
            Thread.sleep(tim);
        }catch(InterruptedException e){}
        System.out.println("我接完"+CallerName+"的电话");
    } }

class Caller implements Runnable{
    String name;
    Receiver r;
    String msg;
    Caller(String name,Receiver r,String msg){
        this.name=name; this.r=r; this.msg=msg;
    }
    public void run(){
        r.call(name,msg); }
}
```



```
class Call{  
    public static void main(String args[])  
    {
```

```
        Receiver r=new Receiver();  
        Caller s1=new Caller("张三",r,"你好吗");  
        Caller s2=new Caller("李四",r,"万事如意");  
        Caller s3=new Caller("王二",r,"新年好");  
        Thread th1=new Thread(s1);  
        Thread th2=new Thread(s2);  
        Thread th3=new Thread(s3);  
        th1.start();  
        th2.start();  
        th3.start();  
        try{  
            th1.join();th2.join();th3.join();  
        }catch(InterruptedException e)  
        {  
        }  
    }
```



复习与预习

复习:

1.多线程两种方法;

预习:

I/O流

